Pico BASIC User's Manual

Pico BASIC for the RP2350 · October 30, 2026

Contents

1. Getting started
   Boards
   Loading it onto the board
   Connecting
   The terminal
   First program
   Editing a line
2. Program and direct commands
3. Language
   Numbers, strings and variables
   Arrays
   Operators, highest priority first
   Flow of control
   Variables and data
   Text output and input
   PRINT USING
   Functions
   Error trapping
   Events
   Memory: PEEK and POKE
4. The flash disk and data files
   The disk
   The SD card (drive SD:)
   Files from programs
5. Moving files to and from the PC
   Pasting and listing
   XMODEM: XRECV and XSEND
6. Sound
7. Hardware: GPIO, PWM, ADC, I2C, SPI, COM1
   Pins
   I2C and SPI
   NeoPixels: WS2812B LEDs
   Serial port (COM1)
   The Cytron Maker Pi Pico
8. Memory
9. Errors
10. Not in this build
11. Limits
12. Example programs

Pico BASIC is a GW-BASIC style interpreter for a bare RP2350 or RP2040 board, used from a serial terminal. It keeps its programs and data files on a FAT disk in the board's own flash, swaps files with the PC by XMODEM, plays tones on a speaker pin, and drives the chip's GPIO, PWM, ADC, I2C, SPI and a second UART. It needs no HDMI, no keyboard and no USB drive: just the board and a USB cable to the PC, where it appears as a serial port.

It is the BASIC of the Pico 2 mini OS, built on its own. The features that need a screen of their own (graphics, sprites, pictures, fonts, the mouse, MOD music) aren't in this build; section 10 lists them.

1. Getting started

Boards

Board	Chip	Flash disk	Memory for BASIC
Raspberry Pi Pico 2	RP2350A, GPIO 0-29	about 3 MB	about 450 KB (SRAM)
Pimoroni Pico Plus 2	RP2350B, GPIO 0-47	about 15 MB	about 8.4 MB (8 MB PSRAM + SRAM)
Raspberry Pi Pico (the original)	RP2040, GPIO 0-29	about 1 MB	about 190 KB (SRAM)

Each board needs its own build: `build/pico_basic.uf2` for the Pico 2, `build-plus2/pico_basic.uf2` for the Pico Plus 2 and `build-pico/pico_basic.uf2` for the Pico (see the README). On the RP2040, BASIC is the same language, but with less memory and no floating-point hardware, so arithmetic-heavy programs run several times slower than on an RP2350, and a 32 KB stack there has no hardware guard (BASIC still checks it itself).

Loading it onto the board

- **BOOTSEL:** hold the BOOTSEL button while plugging the board into USB, then copy the `.uf2` file to the drive that appears. The board restarts into BASIC, and the same USB port becomes its console.
- **picotool:** while BASIC runs, `picotool load -f build/pico_basic.uf2` (or `picotool reboot -f -u`, to get the BOOTSEL drive) reboots it over its USB console port: no button needed.
- **Debug Probe (SWD):** with Raspberry Pi's OpenOCD:

```
OCD=$(ls -d ~/.pico-sdk/openocd/*/ | tail -1)
"$OCD/openocd" -s "$OCD/scripts" -f interface/cmsis-dap.cfg -f target/rp2350.cfg \
    -c "adapter speed 5000" -c "program build-plus2/pico_basic.elf verify reset
exit"
```

Loading a new BASIC keeps the files on the flash disk (it lives above the program). Loading some other program that is bigger than 1 MB, or that uses the flash for itself, overwrites it.

Connecting

The console is the board's own **USB port**: connect it to the PC with a data USB cable (a charge-only cable or a power supply won't do). It appears as a USB serial port: `/dev/ttyACM0` (or the next free number, `ttyACM1` when a Debug Probe is plugged in too) on Linux, a `COM` port on Windows, `/dev/cu.usbmodem...` on a Mac. On Linux, `ls -l /dev/serial/by-id/` shows it as `usb-Raspberry_Pi_Pico_....`.

The only other wiring is optional:

Board pin	Connect to
GP18 (pin 24)	a piezo buzzer or small speaker (through about 100 Ω) to GND, for BEEP, SOUND and PLAY (the Maker Pi Pico's own buzzer)
GP10-15, 3V3, GND	a microSD card slot, as drive `SD:` (section 4; the Maker Pi Pico has one)
SWD (the debug pins)	a Raspberry Pi Debug Probe, for loading programs with OpenOCD (its UART isn't used)

3.3 V levels only on every pin: never connect RS-232 or 5 V signals straight to them.

The terminal

Any terminal program: the speed setting doesn't matter on a USB serial port (115200 8N1 is fine). Use ANSI or VT100, with the **UTF-8** character set, and make the window at least **80 columns by 30 rows** (BASIC's screen; LOCATE takes rows 1-30). For example:

```
picocom /dev/ttyACM1
minicom -D /dev/ttyACM1
```

On Windows, PuTTY or Tera Term on the board's COM port. gtkterm and screen work too, but can't send files by XMODEM (section 5). On Linux your user needs to be in the `dialout` group to open the port.

When the board starts it shows:

```
Pico BASIC 0.1 for the RP2350 (pimoroni_pico_plus2_rp2350)
PSRAM: 8 MB
Flash disk: 15336 KB, 15332 KB free
Pico BASIC (GW-BASIC style). SYSTEM starts it afresh.
8825424 Bytes free
Ok
```

BASIC waits up to 3 seconds at start-up for a terminal; a terminal that connects later (or reconnects) gets the first three lines again and an `Ok`. The very first start-up formats the flash disk (a second or two).

First program

Lines that start with a number are stored as the program; anything else runs straight away:

```
10 FOR I = 1 TO 3
20   PRINT "HELLO"; I
30 NEXT
RUN
SAVE "HELLO"
```

- Typing a line number alone deletes that line; retyping one replaces it.
- **Ctrl-C** stops a running program with `Break in <line>`; `CONT` carries on from there.
- `SYSTEM` (or `BYE`) clears everything and starts BASIC afresh, as after a reset (files on the disk stay).

Editing a line

At the `Ok` prompt and in `INPUT`:

Key	Does
Left, Right, Home, End	move along the line
Backspace, Delete	delete before / at the cursor
Insert	switches between inserting and overwriting
Esc	clears the line
Up, Down	(at `Ok`) bring back the last 8 lines typed
F1-F10	type their soft-key text (`KEY LIST` shows it: F1 `LIST`, F2 `RUN` and Enter, ... as in GW-BASIC)
Ctrl-C	abandons the line

`EDIT n` (or `EDIT .`, the last line typed or the one that failed) puts a program line on the input line to change; Enter stores it. A long line runs on over several rows. Pasting a whole listing into the terminal at the `Ok` prompt enters it line by line.

2. Program and direct commands

Command	What it does
`RUN`	Runs the program from the start (clears variables first).
`RUN line`	Runs from that line.
`RUN "file"`	Loads `file` and runs it.
`LIST [range]`	Lists the program, pausing every 28 lines (any key goes on, Q stops). A range is `n`, `n-`, `-n` or `n-m`.
`LLIST [range]`	Same as LIST (there's no printer).
`NEW`	Deletes the program and its variables.
`CLEAR`	Clears variables and closes files (any arguments are ignored).
`LOAD "file"[,R]`	Loads a program, and with `,R` runs it. `.BAS` is added if the name has no extension. Reads plain text and GW-BASIC tokenised files.
`SAVE "file"[,A]`	Saves as plain text (`,A` is accepted; text is the only format). `.BAS` is added if the name has no extension.
`MERGE "file"`	Adds the lines of a plain-text program to the one in memory (lines with the same number are replaced).
`CHAIN [MERGE] "file" [, [line][, ALL][, DELETE range]]`	Runs another program from a program, keeping open files. Only the variables named in `COMMON` are passed on, or all of them with `ALL`. `CHAIN MERGE` adds the file's lines (after deleting `range`) and keeps all variables.
`COMMON var[, var...]`	Names the variables CHAIN passes on (arrays as `A()`).
`FILES ["pattern"]`	Lists the files in the current folder (`FILES "*.BAS"`), then the disk's free space: `15331328 Bytes free`.
`KILL "file"`	Deletes a file.
`NAME "old" AS "new"`	Renames a file.
`DELETE range`	Deletes program lines.
`RENUM [new][,[old][,step]]`	Renumbers lines (default 10, first, 10), and the GOTO/GOSUB/THEN/ELSE/RESTORE/RUN targets with them.
`AUTO [start][,step]`	Numbers new lines for you (default 10,10). A `*` after the number means that line exists: Enter keeps it. An empty line where none exists, or Ctrl-C, stops.
`EDIT n` / `EDIT .`	Puts line `n` on the input line to edit.
`CONT`	Continues after Ctrl-C or STOP.
`TRON` / `TROFF`	Turns line-number tracing on/off: each line's number is printed in brackets as it runs.
`SYSTEM` / `BYE`	Starts BASIC afresh.

Example:

```
LOAD "PRIMES": LIST 10-50
RENUM 100, , 10
SAVE "PRIMES2"
```

3. Language

Numbers, strings and variables

- **Single precision** (`!`, and names without a type) keep 7 digits, as in GW-BASIC: a value stored in one is rounded to single precision and prints with up to 7 digits.
- **Double precision** (`#`) keep about 15 digits (printed with up to 9); calculations are always done in double precision.
- **Integers** (`%`) hold whole numbers from -32768 to 32767: a value stored in one is rounded (2.5 becomes 3), and one out of range gives Overflow.
- **Constants:** `1.5`, `1E6`, `3#`, `5%`, hex `&HFF`, octal `&O17`.
- **Strings** (`$`) are up to 255 characters and may hold any byte, including CHR$(0).
- **Names** are up to 40 characters: letters, digits and `.`, starting with a letter, with an optional type character. `A$`, `A%`, `A!` and `A#` are four different variables, and `A` is `A!` unless `DEFINT`/`DEFSNG`/ `DEFDBL`/`DEFSTR` say otherwise (after `DEFINT A-Z`, `A` is `A%`).
- An array and a plain variable may share a name: `A` and `A(3)` are separate.
- Keywords are recognised without spaces (GW-BASIC style), so a variable name must not start with a keyword: `PRINTER` reads as `PRINT ER`.

Arrays

- `DIM A(20), B$(5,5)`: indices start at 0; up to 8 dimensions.
- An array used without DIM gets 0-10 in each dimension.
- `OPTION BASE 1` (before any array is used) makes subscripts start at 1.
- `ERASE A, B$` deletes arrays, so they can be dimensioned again.

Operators, highest priority first

Operators	Meaning
`^`	power
`-`	negation
`*` `/`	multiply, divide
`\`	integer division
`MOD`	remainder
`+` `-`	add, subtract (`+` also joins strings)
`=` `<>` `<` `>` `<=` `>=`	comparisons, giving -1 for true and 0 for false (strings compare too)
`NOT`	
`AND`	
`OR` `XOR`	
`EQV` `IMP`	

The logical operators work bit by bit on integers, as in GW-BASIC.

Flow of control

Statement	Notes
`GOTO line`	`GOTO 100`
`GOSUB line` … `RETURN`	RETURN ends the FOR and WHILE loops the subroutine started.
`ON n GOTO l1, l2, ...` / `ON n GOSUB ...`	Goes to the n-th line; past the end, carries on.
`IF cond THEN statements [ELSE statements]`	`THEN 100` and `ELSE 200` jump to lines; `IF X GOTO 100` works too. `IF A > 0 THEN PRINT "+" ELSE PRINT "-"`
`FOR v = a TO b [STEP s]` … `NEXT [v[, v...]]`	`FOR I = 10 TO 1 STEP -1: PRINT I: NEXT`
`EXIT FOR`	Leaves the innermost FOR loop (from QuickBASIC).
`WHILE cond` … `WEND`	`WHILE INKEY$ = "": WEND` waits for a key.
`END`	Ends the program (closes files).
`STOP`	Stops with `Break in <line>`; CONT goes on.
`PAUSE ms`	Waits (an extension); Ctrl-C still works. `PAUSE 500`
`REM text` or `' text`	A remark.

Several statements can share a line, separated by `:`.

Variables and data

Statement	Notes
`LET var = expr`	`LET` is optional: `X = 5`.
`DIM`, `ERASE`, `OPTION BASE`	See Arrays.
`DEFINT`, `DEFSNG`, `DEFDBL`, `DEFSTR letters`	`DEFINT A-Z`, `DEFSTR S, T`. RUN, NEW and CLEAR reset them.
`DATA items`, `READ var[, var...]`, `RESTORE [line]`	`10 DATA 3, "RED", 2.5` / `20 READ N, C$, X`
`SWAP a, b`	Exchanges two variables of the same type.
`DEF FNname[(args)] = expr`	`DEF FNSQ(X) = X * X`, then `PRINT FNSQ(3)`.
`MID$(v$, start[, n]) = x$`	Replaces characters of `v$` in place (its length never changes).
`RANDOMIZE [seed]`	Without a seed it uses the clock.
`DATE$ = "mm-dd-yyyy"`, `TIME$ = "hh:mm:ss"`	Sets the clock (`-` or `/` and two-digit years are accepted).
`ENVIRON "NAME=value"`	Sets a string `ENVIRON$` returns (there's no DOS environment).

The board has no battery-backed clock: at power-up the clock starts at 01-01-1970 00:00:00 and counts from there, until `DATE$` and `TIME$` set it. Times are UTC. Files saved before it is set are dated 1 January 2026.

Text output and input

Statement	Notes
`PRINT items` or `? items`	`;` keeps the cursor on the line, `,` jumps to the next 14-column zone; `TAB(n)` and `SPC(n)`. `PRINT "X ="; X, "Y ="; Y`
`PRINT USING fmt$; values`	Formatted numbers and strings (below). `PRINT USING "##.##"; 3.14159`
`LPRINT`, `LPRINT USING`	Same as PRINT (there's no printer).
`WRITE items`	Items separated by commas, strings in quotes.
`INPUT ["prompt";] var[, var...]`	With `,` instead of `;` after the prompt, no `?` is shown. Asks again if the input doesn't fit.
`LINE INPUT ["prompt";] var$`	Reads a whole line, commas included.
`CLS`	Clears the screen.
`LOCATE [row][, col][, cursor]`	Rows 1-30, columns 1-80; cursor 0 hides it, 1 shows it. `LOCATE 10, 30: PRINT "HERE"`
`COLOR [fg][, bg]`	PC colours 0-15 (fg 16-31 counts as 0-15), sent as ANSI colours; 7 (fg) and 0 (bg) give the terminal's own. `COLOR 14, 1`
`VIEW PRINT [top TO bottom]`	Text scrolls only within rows top-bottom; `VIEW PRINT` alone undoes it.
`KEY n, x$`, `KEY LIST`, `KEY ON` / `OFF`	Sets or lists the soft keys F1-F10. `KEY ON` shows their labels on row 25, as on a PC. `KEY 5, "FILES" + CHR$(13)`
`WIDTH n`	Accepted; the screen stays 80 columns.

Characters: BASIC uses the IBM PC character set (code page 437), and sends each character to the terminal as its Unicode equivalent in UTF-8: CHR$(219) is a solid block, CHR$(176)-CHR$(178) are shades, CHR$(196) and CHR$(179) are box lines, and control codes show their pictures (CHR$(1) is a smiley, CHR$(3) a heart), except 7 (beep), 8, 9, 10, 12 (clear screen), 13 and 27. 11 moves the cursor home and 28-31 move it right, left, up and down. Text you type is stored as the bytes the terminal sends (UTF-8 for accented letters).

PRINT USING

`PRINT USING fmt$; a, b, ...` prints the values in the fields of `fmt$` (used again from the start if there are more values; other characters are printed as they are):

Field	Meaning
`#`	A digit position. `##.##` prints `3.5` as `3.50`.
`.`	The decimal point.
`,` before the point	Commas between thousands (it also counts as a digit).
`+` first or last	Always show the sign there.
`-` last	A minus after negative numbers.
`**`	Fill the space before the number with `*`.
`$$`	A `$` just before the number. `**$` does both.
`^^^^` (or `^^^^^`)	Exponent form: `##.##^^^^` prints `2.35E+02`.

Field	Meaning
`!`	The first character of a string.
`\ \`	The first n + 2 characters of a string (n spaces between).
`&`	The whole string.
`_`	Prints the next character as it is.

A number too big for its field is printed in full after a `%`. Also `PRINT #n, USING` and after other items: `PRINT "X ="; USING "##.#"; X`.

Functions

Kind	Functions
Maths	`ABS(x)` `INT(x)` (rounds down) `FIX(x)` (cuts off the fraction) `SGN(x)` `SQR(x)` `SIN(x)` `COS(x)` `TAN(x)` `ATN(x)` `EXP(x)` `LOG(x)` `CINT(x)` `CSNG(x)` `CDBL(x)`
Random	`RND` or `RND(1)`: the next number (0 to 1); `RND(0)` repeats the last one; `RND(-n)` reseeds. RUN always starts the same sequence: use `RANDOMIZE` for a different one.
Strings	`LEN(s$)` `LEFT$(s$,n)` `RIGHT$(s$,n)` `MID$(s$,start[,n])` `CHR$(n)` `ASC(s$)` `STR$(x)` `VAL(s$)` `INSTR([start,] s$, find$)` `STRING$(n, c or c$)` `SPACE$(n)` `HEX$(n)` `OCT$(n)` `UCASE$(s$)` `LCASE$(s$)`
Keyboard	`INKEY$`: the key waiting, or "". Cursor and function keys give CHR$(0) and the PC scan code: Up CHR$(0)+CHR$(72), Down 80, Left 75, Right 77, F1-F10 59-68, Home 71, End 79, PgUp 73, PgDn 81, Ins 82, Del 83. `INPUT$(n)`: reads n keys without showing them.
Screen	`POS(0)` (cursor column), `CSRLIN` (cursor row)
System	`TIMER` (seconds since start-up), `FRE(0)` (free memory), `DATE$`, `TIME$`, `ENVIRON$("name")` or `ENVIRON$(n)`, `PEEK(offset)`
Errors	`ERR`, `ERL` (see Error trapping), `ERDEV`, `ERDEV$`, `EXTERR(n)` (always 0 / "")
Sound	`PLAY(0)`: notes still waiting in the background queue
Files	`EOF(n)` `LOF(n)` `LOC(n)` `INPUT$(count, [#]n)` `FREEFILE` `DIR$(pattern$[, n])` `MKI$` `MKS$` `MKD$` `CVI` `CVS` `CVD` (section 4)
Hardware	`PIN(pin)`, `ADC(pin)`, `TEMP()`, `I2CREAD(...)`, `I2CREAD$(...)`, `SPI$(...)` (section 6)
Old or PC-only	`LPOS(0)`, `STICK(n)`, `STRIG(n)`, `PEN(n)` give 0; `INP`, `USR`, `VARPTR`, `VARPTR$`, `IOCTL$` are an Illegal function call

Examples:

```
PRINT INT(-2.5), FIX(-2.5), CINT(2.5)        ' -3  -2  3
PRINT INSTR("HELLO", "L"), MID$("HELLO", 2, 3) '  3  ELL
PRINT HEX$(255); " "; STRING$(5, "*")        ' FF *****
K$ = INKEY$: IF K$ = CHR$(0) + CHR$(72) THEN PRINT "UP"
```

Error trapping

Statement	Notes
`ON ERROR GOTO line`	Sends errors in a running program to that line; `ON ERROR GOTO 0` turns it off (inside the handler, it reports the error and stops).

Statement	Notes
`RESUME`, `RESUME NEXT`, `RESUME line`	Ends the handler: retries the statement that failed, goes on after it, or goes to the line.
`ERROR n`	Raises error `n` (1-255): a test, or a program's own error codes.

```
10 ON ERROR GOTO 100
20 OPEN "DATA.TXT" FOR INPUT AS #1
30 PRINT "opened": END
100 IF ERR = 53 THEN PRINT "no file": RESUME 30
110 ON ERROR GOTO 0
```

A program that reaches its end while handling an error stops with `No RESUME`. Ctrl-C can't be trapped.

Events

`ON KEY`, `ON TIMER`, `ON PLAY` and `ON COM` jump to a subroutine when something happens; they are checked between statements. `ON`, `OFF` and `STOP` switch an event: one that happens while it is STOPped (or while its own subroutine runs) is remembered and handled once it is back ON. RETURN from the subroutine continues the program where it was.

Statement	When
`ON KEY(n) GOSUB line`, `KEY(n) ON` / `OFF` / `STOP`	Key `n` is pressed: 1-10 are F1-F10, 11 Up, 12 Left, 13 Right, 14 Down.
`ON TIMER(s) GOSUB line`, `TIMER ON` / `OFF` / `STOP`	Every `s` seconds (1-86400).
`ON PLAY(n) GOSUB line`, `PLAY ON` / `OFF` / `STOP`	The background music queue drops below `n` notes.
`ON COM(1) GOSUB line`, `COM(1) ON` / `OFF` / `STOP`	Something arrives on the open COM1.

```
10 ON TIMER(1) GOSUB 100: TIMER ON
20 ON KEY(1) GOSUB 200: KEY(1) ON
30 GOTO 30
100 PRINT TIME$: RETURN
200 PRINT "F1 pressed": RETURN
```

Memory: PEEK and POKE

There's no PC memory. `DEF SEG = segment` with `PEEK(offset)`, `POKE offset, byte`, `BSAVE "file", offset, length` and `BLOAD "file"[, offset]` reach two emulated places; anything else is an Illegal function call:

- `DEF SEG = &HB800` (or `&HB000`): the text screen, two bytes a character. A terminal can't be read back, so PEEK gives spaces (32) and colour 7, and POKE does nothing.
- `DEF SEG = 0` (or `&H40`): BIOS bytes: video mode (&H449, 3), columns (&H44A, 80), rows - 1 (&H484, 29) and the timer tick count (&H46C-&H46E, 18.2 a second, from midnight). The rest read 0, and POKEs there do nothing.

`SCREEN(row, col[, 1])` likewise gives 32 (and with 1, colour 7).

4. The flash disk and data files

The disk

The flash disk is a FAT file system in the board's flash, after the program: about 3 MB on a Pico 2 and 15 MB on a Pico Plus 2. It is formatted (labelled PICOBASIC) the first time the board starts, and keeps its files when the board is switched off or BASIC is loaded again.

- `FILES` lists it and ends with the free space; `FRE(0)` is memory, not disk.
- Names are FAT long names (case is kept, but matching ignores it). Folders work: `MKDIR "GAMES"`, `CHDIR "GAMES"`, `CHDIR ".."`, `RMDIR "GAMES"`, and paths such as `LOAD "GAMES/GUESS"`.
- **Writes are kept in a 4 KB cache** and go to the flash when a file is closed (or the cache moves on to another part of the disk). SAVE closes its file at once; a program's own files are closed by CLOSE, END, RUN, NEW, an error that stops it, and so on. **CLOSE files before switching off**, or the last few KB written can be lost.
- The flash wears out after about 100,000 writes to the same place: plenty for saving programs and logging now and then, but don't rewrite a file many times a second for hours.
- While the flash is being written (a few tens of milliseconds for each 4 KB) the board does nothing else; background PLAY notes may stretch.

The SD card (drive SD:)

A microSD card on SPI is a second drive, `SD:` (or `1:`); the flash disk is `FLASH:` (or `0:`) and stays the current drive at start-up. Cards formatted FAT16, FAT32 (SDHC, up to 32 GB) or exFAT (SDXC) work, as a PC formats them; BASIC doesn't format cards.

Wiring, for the usual microSD breakout (one with its own 3.3 V regulator and level shifter is fine too, powered from 3V3 or VSYS as it says):

Card / module	Pico pin
CLK (SCK)	GP10 (pin 14)
SDI / DI (MOSI)	GP11 (pin 15)
SDO / DO (MISO)	GP12 (pin 16)
DAT1, DAT2 (if the slot has them)	GP13, GP14 (held up: unused in SPI mode)
CS (DAT3)	GP15 (pin 20)
3V3 (VCC)	3V3(OUT) (pin 36)
GND	GND (pin 18)

These are SPI1's pins, and the Cytron Maker Pi Pico's microSD slot is wired just so: it works as it is. Others can be chosen when BASIC is built (see the README). The banner shows the card: `SD card (SD:): 7580 MB, 7571 MB free`, or `SD card (SD:): none`.

- Use it with a drive in front of a name: `SAVE "SD:GAME"`, `LOAD "SD:/GAMES/GUESS"`, `OPEN "SD:LOG.TXT" FOR APPEND AS #1`, `FILES "SD:"`, `FILES "SD:/GAMES/*.BAS"`, `XRECV "SD:PIC.BMP"`.
- `CHDIR "SD:"` makes the card the current drive (then names without a drive are on the card), `CHDIR "SD:/GAMES"` its folder too, and `CHDIR "FLASH:"` goes back.
- Copying between drives: `LOAD "SD:GAME"` then `SAVE "FLASH:GAME"`; for any file, a program that reads one with GET and writes the other with PUT.
- A card can be put in or taken out while BASIC runs: the next use of `SD:` finds the new one. Close files on it first (CLOSE) and don't pull it out while it is being written.
- While a card is in, its pins (GP10-15) and SPI1 are BASIC's: `PIN`, `PWM` and `SPI OPEN` on SPI1 refuse them (`SPI OPEN` on SPI0, its default, is fine). Without a card they are free.

Files from programs

Up to 4 files can be open at once, numbered #1 to #4. File names are relative to the current folder; no extension is added. Each open file uses about 0.8 KB of memory (plus its record buffer for random files).

Statement / function	What it does
`OPEN "file" FOR INPUT | OUTPUT | APPEND | RANDOM AS [#]n [LEN=size]`	Opens a file. OUTPUT creates or empties it, APPEND adds to the end, RANDOM (also the default when FOR is left out) reads and writes fixed-size records (LEN 1-1024, default 128).
`OPEN "I" | "O" | "A" | "R", [#]n, "file"[, size]`	The older short form.
`CLOSE [[#]n[, [#]n...]]`, `RESET`	Close the given files, or all of them.
`PRINT #n, items`, `PRINT #n, USING fmt$; items`	Like PRINT, into the file. Lines end in CR LF, with no 80-column wrap.
`WRITE #n, items`	Items separated by commas, strings in quotes, so INPUT # reads them back exactly.
`INPUT #n, var[, var...]`	Reads items: a quoted string, or text up to a comma or the end of the line (numbers also stop at a space).
`LINE INPUT #n, var$`	Reads a whole line.
`INPUT$(count, [#]n)`	Reads count characters.
`EOF(n)`	-1 when there is nothing more to read (for a random file: the last GET was past the end).
`LOF(n)`	The file's length in bytes.
`LOC(n)`	For a random file, the last record read or written; otherwise the position in 128-byte blocks.
`FREEFILE`	The lowest free file number.
`FIELD #n, width AS var$[, width AS var$...]`	Divides a random file's record buffer into string fields (at most 16, and no more than the record length).
`LSET var$ = s$` / `RSET var$ = s$`	Puts a string into a field, padded with spaces, aligned left or right. Use these, not LET, for field variables.
`PUT [#]n[, record]`	Writes the record buffer as record `record` (from 1; default: the one after the last).
`GET [#]n[, record]`	Reads a record into the buffer and its FIELD variables.

Statement / function	What it does
`MKI$(n)` / `CVI(s$)`	A whole number -32768 to 32767 as a 2-byte string (low byte first), and back.
`MKS$(x)` / `CVS(s$)`	A number as a 4-byte single-precision string, and back.
`MKD$(x)` / `CVD(s$)`	A number as an 8-byte double-precision string, and back.
`DIR$(pattern$[, n])`	The n-th name (1 first) matching the pattern, e.g. `DIR$("*.BAS", 2)`; "" after the last. For going through a folder's files.

Files stay open across lines and statements until CLOSE. RUN, NEW, CLEAR, END, the program finishing, an error that stops the program, and SYSTEM close every file. Ctrl-C does not, so CONT can carry on.

```
10 OPEN "SCORES.TXT" FOR OUTPUT AS #1
20 WRITE #1, "Ann", 120
30 CLOSE #1
40 OPEN "SCORES.TXT" FOR INPUT AS #1
50 WHILE NOT EOF(1): INPUT #1, N$, S: PRINT N$, S: WEND
60 CLOSE #1
```

Random access (a 22-byte record: a 20-character name and a 2-byte number):

```
10 OPEN "PEOPLE.DAT" FOR RANDOM AS #1 LEN = 22
20 FIELD #1, 20 AS NM$, 2 AS AGE$
30 LSET NM$ = "Grace": LSET AGE$ = MKI$(85): PUT #1, 1
40 GET #1, 1: PRINT NM$; CVI(AGE$)
50 CLOSE #1
```

MKS$ and MKD$ use GW-BASIC's own number format (Microsoft Binary Format), so random files written by GW-BASIC on a PC read correctly here, and the other way round. CVI, CVS and CVD need at least 2, 4 and 8 bytes (otherwise Illegal function call).

Differences from GW-BASIC:

- A file can be open for INPUT under several numbers, but not written while it is open elsewhere.
- Ctrl-Z (byte 26) ends a file for INPUT # and INPUT$, as in GW-BASIC, so binary files are best read with random access (GET).

5. Moving files to and from the PC

Pasting and listing

The simplest way to bring a program over: type `NEW`, paste its listing into the terminal at the `Ok` prompt (the numbered lines go into the program), then `SAVE "NAME"`. To take one back: `LOAD` it, `LIST` it, and copy the text from the terminal window. Many terminals can also log what is shown to a file.

If pasted lines get lost, slow the terminal's paste down (a few milliseconds per line): BASIC reads keys between lines, and the USB port holds only a few hundred characters meanwhile.

XMODEM: XRECV and XSEND

These copy any file, text or binary, between the flash disk and the PC over the terminal line:

Statement	What it does
`XRECV "file"`	Receives a file from the terminal's XMODEM send and saves it as `file`, replacing a file of that name. It asks for CRC-16, falling back to checksums for an old sender, and takes 128- or 1K-byte blocks. XMODEM pads the last block with ^Z (byte 26) characters; they are taken off the end. If the transfer fails, the partial file is deleted (a file that was already there is kept if nothing arrived).
`XSEND "file"`	Sends the file to the terminal's XMODEM receive, in 1K blocks with CRC-16, or 128-byte blocks with a checksum if the receiver asks for those.

Type the command; BASIC says `Start the XMODEM send on the terminal now.` (or receive), and you then have a minute to start the transfer on the PC. XMODEM doesn't send the file's name, so give it on both ends. Pressing Ctrl-X a few times in the terminal cancels a transfer; a failed one gives `Device I/O error` after a line saying why. When it ends, BASIC prints e.g. `Received 1 file, 329 bytes.`

```
XRECV "GAME.BAS"
(send GAME.BAS from the terminal)
LOAD "GAME"
RUN
```

Starting the PC's side:

- **minicom:** Ctrl-A S, choose `xmodem`, then the file, to send it to XRECV. Ctrl-A R, `xmodem` and a name receives from XSEND. (minicom uses lrzsz's `sx` and `rx`: `sudo apt install lrzsz`.)
- **picocom:** start it with `picocom --send-cmd "sx -vv" --receive-cmd "rx -vv" /dev/ttyACM1`, then Ctrl-A Ctrl-S sends a file and Ctrl-A Ctrl-R receives one (type the name to save it as).
- **Tera Term:** File > Transfer > XMODEM > Send... or Receive... (pick CRC or 1K).
- **From a shell**, with the terminal program closed (or disconnected) so it doesn't take the bytes:

```
stty -F /dev/ttyACM1 raw -echo
sx GAME.BAS < /dev/ttyACM1 > /dev/ttyACM1      # to XRECV
rx GAME.BAS < /dev/ttyACM1 > /dev/ttyACM1      # from XSEND
```

(with the board's port in place of `/dev/ttyACM1`).

The PC's receiver may keep XMODEM's ^Z padding (up to 1 KB of byte 26 at the end). lrzsz's `rx` does, unless run as `rx -a` for a text file (which also converts line ends). Because BASIC removes it, a binary file that really ends in ^Z bytes loses them on the way in.

6. Sound

A square wave on **GP18** (another pin if BASIC was built with a different `SOUND_PIN`), for a piezo buzzer or a small speaker through a resistor.

Statement	What it does
`BEEP`	A short beep (800 Hz for a quarter of a second).
`SOUND freq, ticks`	A tone of freq Hz (37-32767; 32767 is silence) for `ticks` (18.2 make a second). After `PLAY "MB"` it goes to the background queue; ticks 0 stops the sound playing. `SOUND 440, 18`
`PLAY "string"`	Music in GW-BASIC's music language (below). `PLAY "T160 O3 L8 CDEFGAB > C"`
`PLAY(0)`	Notes still waiting in the background queue.

PLAY's commands:

Command	Meaning
`A-G` [`#`, `+` or `-`] [length] [`.`]	A note (sharp or flat), optionally with its own length; each dot adds half again.
`N n`	Note number n (1-84; 0 is a rest).
`O n`, `>`, `<`	Octave 0-6 (default 4; octave 3 starts at middle C), up one, down one.
`L n`	Length for following notes: 1 whole, 4 quarter (default), up to 64.
`P n`	A pause, length as for L.
`T n`	Tempo, 32-255 quarter notes a minute (default 120).
`MN` `ML` `MS`	Normal (7/8 of each note sounds), legato (all), staccato (3/4).
`MF` `MB`	Foreground (PLAY waits until the music ends; the default) or background (up to 32 notes queue up and the program goes on).
`=variable;`	A number from a variable: `PLAY "O=OC;C"`.

The settings last from one PLAY to the next; RUN resets them. `X` (substrings through VARPTR$) isn't supported. The tones are plain square waves of one voice, so a note waits for the one before it.

7. Hardware: GPIO, PWM, ADC, I2C, SPI, COM1

Pins

Pins are given by their GP number; `LED` means the board's LED (GP25). The Pico and Pico 2 have GP0-29 (GP23-25 and GP29 aren't on the header: power control, VBUS sense, the LED and VSYS / 3); the Pico Plus 2 has GP0-47.

Pins BASIC itself uses are refused, e.g. `GP18 is used by the speaker (BEEP, SOUND, PLAY)` followed by `Illegal function call`:

Pins	Used by
GP18	the speaker
GP10-15	the SD card, while one is in
GP47 (Pico Plus 2)	the PSRAM
COM1's, I2C's and SPI's pins	while they are open

Statement / function	What it does
`PINMODE pin, INPUT | OUTPUT | PULLUP | PULLDOWN`	Sets a pin up (or modes 0-3). `PINMODE 16, PULLUP`
`PIN pin, level`	Drives a pin 0 or 1, making it an output first if needed. `PIN LED, 1`
`PIN(pin)`	Reads a pin, making an unused pin an input. `IF PIN(16) = 0 THEN PRINT "PRESSED"`
`PWM pin, freq[, duty]`	PWM at 8 Hz-10 MHz with a duty of 0-100 % (default 50). `PWM pin, 0` stops it. `PWM 16, 1000, 25`
`SERVO pin, angle`	A hobby servo, 0-180 degrees (0.5-2.5 ms pulses at 50 Hz). `SERVO 16, 90`
`ADC(pin)`	An analog input, 0-4095 for 0-3.3 V: GP26-29 on the Pico and Pico 2, GP40-46 on the Pico Plus 2. `V = ADC(26) * 3.3 / 4095`
`TEMP()`	The chip's temperature in °C (uncalibrated: within a few degrees).
`PINS`	Lists every pin and what it is doing.
`NEOPIXEL ...`	WS2812B LEDs (below).

- PWM pins come in pairs that share one frequency (GP0/1, GP2/3 and so on); changing one's frequency prints a warning naming its partner. The chip has only 8 slices for GP0-31, so GP16-31 use the same ones as GP0-15: GP2 and GP18 (and GP3 and GP19) give the very same signal, with no warning. The speaker's slice (GP18) is shared with GP2, GP3 and GP19: PWM there and sound interfere.
- `RUN`, `NEW` and `SYSTEM` set every pin BASIC used back to an input and stop PWM. `END`, errors and Ctrl-C leave pins as they are, so they can be checked in direct mode.
- `PIN`, `ADC` and `TEMP` followed by `(` are functions, so arrays can't use those names.

I2C and SPI

The chip's own I2C and SPI controllers, for sensors, displays, memory chips and the like. The pins choose the controller: I2C wants SDA on an even pin and SCL on the next one (I2C0 or I2C1 by the pin numbers); SPI wants SCK, MOSI and MISO from one group of four (MISO, CS, SCK, MOSI: GP8-11 is SPI1, GP16-19 SPI0 ...). RUN, NEW and SYSTEM close them.

Statement / function	What it does
`I2C OPEN [sda, scl[, khz]]`	Starts I2C, by default on **GP2 (SDA) and GP3 (SCL)**, I2C1, at 100 kHz (10-1000). Turns on the pins' weak pull-ups; devices should have their own.
`I2C WRITE addr, item[, item...][, NOSTOP]`	Sends bytes to the device at `addr` (0-127): numbers 0-255 and strings (their bytes). With `NOSTOP` the bus is kept for a read that follows.
`I2CREAD$(addr, n[, reg])`	Reads `n` bytes (1-255); with `reg`, first writes that register number (without a stop).
`I2CREAD(addr[, reg])`	Reads one byte.
`I2C SCAN`	Prints the addresses that answer (8-119).
`I2C CLOSE`	Stops it and frees the pins.
`SPI OPEN [sck, mosi, miso[, khz[, mode]]]`	Starts SPI, by default on **GP6 (SCK), GP7 (MOSI), GP4 (MISO)**, SPI0, at 1000 kHz (1-50000), mode 0 (0-3); 8-bit words, most significant bit first. Chip select is any pin you drive with `PIN` (**GP5** is free for it). (SPI1 is the SD card's.)
`SPI WRITE item[, item...]`	Sends bytes (numbers 0-255 and strings), ignoring what comes back.
`SPI$(out$)`	Sends `out$` and returns the bytes that came back meanwhile: `SPI$(STRING$(4, 0))` reads 4.
`SPI CLOSE`	Stops it and frees the pins.

A device that doesn't answer gives `Device I/O error` (57); using I2C or SPI before OPEN gives `Device unavailable` (68). Both can be caught with ON ERROR.

```
10 I2C OPEN: I2C SCAN
20 T = I2CREAD(&H48, 0): PRINT "LM75 at &H48:"; T
30 PIN 5, 1: SPI OPEN: PIN 5, 0: SPI WRITE &H9F
40 ID$ = SPI$(STRING$(3, 0)): PIN 5, 1: PRINT LEN(ID$); "bytes"
```

NeoPixels: WS2812B LEDs

Strips, rings and single WS2812B ("NeoPixel") LEDs, on any free pin, driven by one of the chip's PIO state machines. Colours are kept in a buffer and sent with `NEOPIXEL SHOW`.

Statement	What it does
`NEOPIXEL OPEN [pin[, count[, order$]]]`	Sets up `count` LEDs (1-1024, default 1) on `pin` (default **GP28**, the Maker Pi Pico's RGB LED), all off. `order$` is `"GRB"` (WS2812B, the default) or `"RGB"` for strips that take red first. Opening again replaces the strip.
`NEOPIXEL n, r, g, b`	LED `n` (0 first) to red, green and blue 0-255, in the buffer.
`NEOPIXEL FILL r, g, b`	Every LED to one colour.

Statement	What it does
`NEOPIXEL SHOW`	Sends the buffer to the LEDs.
`NEOPIXEL BRIGHT level`	Brightness 0-255 (255 at first) for the next SHOWs: full white is bright and draws 60 mA a LED.
`NEOPIXEL CLEAR`	All off, shown at once.
`NEOPIXEL CLOSE`	Frees the pin and the state machine.

- The pin is refused to `PIN`, `PWM` and the rest while the strip is open; `PINS` shows it. RUN, NEW and SYSTEM close it (the LEDs keep their last colours: `NEOPIXEL CLEAR` first to turn them off).
- Using it before `NEOPIXEL OPEN` gives `Device unavailable` (68).
- WS2812B LEDs want 5 V but take the Pico's 3.3 V data on a short wire. Power a strip of more than a few LEDs from its own 5 V supply, join its GND to the Pico's, and keep `BRIGHT` modest.

  10 NEOPIXEL OPEN 16, 8: NEOPIXEL BRIGHT 40 20 FOR I = 0 TO 7: NEOPIXEL I, I * 32, 0, 255 - I * 32: NEXT 30 NEOPIXEL SHOW

`examples/NEOPIXEL.BAS` shows red, green and blue, then a rainbow.

Serial port (COM1)

A serial port on the chip's UART1 (the console is on USB). It works like a file: open it with a "COM1:" name, then use PRINT #, WRITE #, INPUT #, LINE INPUT # and INPUT$ on it.

```
OPEN "COM1:[speed][,parity][,data][,stop][,options]" AS #n
```

- **Settings:** speed 75-921600 baud; parity N, E or O; data bits 5-8; stop bits 1 or 2. As in GW-BASIC, anything left out defaults to 300 baud, even parity, 7 data bits and 1 stop bit, so usually you give them all: `OPEN "COM1:9600,N,8,1" AS #1`.
- **Pins:** GP8 (TX) and GP9 (RX) unless `TXn` / `RXn` options choose others: TX on GP4, 8 or 20 and RX on GP5, 9 or 21 (`OPEN "COM1:9600,N,8,1,TX8,RX9" AS #1`). While it is open, those pins are refused to PIN and PWM.
- **Options:** `LF` sends an LF after each CR. `RS`, `CS`, `DS`, `CD`, `PE`, `BIN` and `ASC` are accepted and do nothing (there are no handshake lines).
- **Sending:** PRINT # ends lines with CR (CR LF with LF). It waits while the UART is busy.
- **Receiving:** bytes go into a 4 KB buffer in the background, so none are lost while the program is busy. `LOC(n)` is how many are waiting, `EOF(n)` is -1 when none are, and `LOF(n)` is the room left. INPUT #, LINE INPUT # and INPUT$ wait for data (Ctrl-C stops the wait). A line ends at CR (an LF after it is skipped). `ON COM(1) GOSUB` runs when bytes arrive.
- **Levels:** 3.3 V logic. Use a level shifter or an RS-232 adapter for RS-232 or 5 V devices, and connect GND too.
- Only one COM port can be open, and GET/PUT/FIELD don't apply to it.

`examples/SERIAL.BAS` is a small terminal: wire GP8 to GP9 and what you type comes straight back.

The Cytron Maker Pi Pico

BASIC's defaults fit this carrier board:

- **microSD slot:** drive `SD:` as it is (SPI1, GP10-15).
- **Buzzer:** GP18, for BEEP, SOUND and PLAY (its mute switch turns it off); the 3.5 mm audio jack is on GP18 (left) and GP19 (right).
- **Buttons:** GP20, GP21 and GP22, with their own pull-ups: `PIN(20)` is 0 while the button is pressed.
- **RGB LED:** a WS2812B on GP28: `NEOPIXEL OPEN` (with no arguments) then `NEOPIXEL 0, 255, 0, 0: NEOPIXEL SHOW` lights it red.
- **LEDs:** every GPIO has one, so they show what the pins are doing.
- **Free for your own use:** COM1 (GP8/GP9), `SPI OPEN` (GP4-7), I2C (GP2/GP3) and the Grove connectors on those pins.

8. Memory

- **Pico Plus 2:** the PSRAM is found at start-up (the banner shows it). Blocks of 512 bytes or more (arrays, mostly) go in the 8 MB PSRAM, and small ones (strings, program lines, variables) in the faster SRAM until it runs low, so programs get about 8.4 MB.
- **Pico 2:** everything is in SRAM, about 450 KB.
- **Pico (RP2040):** everything is in SRAM, about 190 KB: room for a long program, but `DIM A(20000)` (8 bytes an element) is about the most one array can have.
- `FRE(0)` (and the banner) give the memory free. A program line is refused if it would leave less than 16 KB for data, and any allocation if it would leave less than 4 KB.
- BASIC runs on a 32 KB stack: very deeply nested expressions give `Out of memory` rather than crashing.

9. Errors

Errors print as `<message> in <line>` (or just the message in direct mode), and `ERR` gives the code:

Code	Message	Code	Message
1	NEXT without FOR	26	FOR without NEXT
2	Syntax error	27	Out of paper
3	RETURN without GOSUB	29	WHILE without WEND
4	Out of DATA	30	WEND without WHILE
5	Illegal function call	50	FIELD overflow
6	Overflow	51	Internal error
7	Out of memory	52	Bad file number
8	Undefined line number	53	File not found
9	Subscript out of range	54	Bad file mode
10	Duplicate Definition	55	File already open
11	Division by zero	57	Device I/O error
12	Illegal direct	58	File already exists
13	Type mismatch	61	Disk full
14	Out of string space	62	Input past end
15	String too long	63	Bad record number
16	String formula too complex	64	Bad file name
17	Can't continue	66	Direct statement in file
18	Undefined user function	67	Too many files
19	No RESUME	68	Device unavailable
20	RESUME without error	69	Communication buffer overflow
22	Missing operand	70	Permission denied
23	Line buffer overflow	71	Disk not ready
24	Device Timeout	72	Disk Media Error
25	Device Fault	73	**Feature not available** (section 10)
		74	Rename across disks
		75	Path/File Access Error
		76	Path not found

Codes BASIC itself never raises can still be raised with `ERROR n`; unknown codes print `Unprintable error`. Ctrl-C prints `Break in <line>`.

10. Not in this build

A terminal has no pixels, so these give **Feature not available** (error 73):

- **Graphics:** `SCREEN` 1 and up (`SCREEN 0` is fine), `PSET`, `PRESET`, `LINE (x,y)-(x,y)`, `CIRCLE`, `PAINT`, `DRAW`, `PALETTE`, `VIEW (...)`, `WINDOW`, graphics `GET (x,y)-(x,y)` / `PUT (x,y)`, `PCOPY`, and the functions `POINT` and `PMAP`.
- **Sprites, fonts and pictures:** `SPRITE`, `SPRITEHIT`, `SPRITEX`, `SPRITEY`, `SPRITEON`, `FONT`, `GPRINT`, `GWIDTH`, `FONTW`, `FONTH`, `IMAGE`, `SAVEIMAGE`.
- **MOD/XM music and the mouse:** `MUSIC` (the functions `MUSIC(n)`, `MOUSEX`, `MOUSEY`, `MOUSEB` and `MOUSEW` just give 0), `MOUSE ON/OFF`.
- **The full-screen editor** (`EDIT` with no line number) and `SHELL`: there is no editor or shell; `EDIT n` works.

Also:

- `KEYDOWN(n)` is always 0: a terminal sends key presses, not whether a key is held.
- `SCREEN(row, col)` and PEEK of the text screen give spaces (section 3, Memory).
- Machine code and PC hardware: `CALL`, `USR`, `DEF USR`, `VARPTR`, `VARPTR$`, `INP`, `OUT`, `WAIT`, `IOCTL` are an Illegal function call.
- Joystick and light pen (`STICK`, `STRIG`, `PEN` give 0), the printer (LPRINT and LLIST go to the screen), COM ports other than COM1, 40-column text, and `LOCK`, `UNLOCK` and `MOTOR` (accepted, they do nothing).
- GW-BASIC's full-screen editing of any line on the screen: lines are edited on the input line instead (`EDIT n`).

11. Limits

- **Lines and strings:** program lines and strings are up to 255 characters.
- **Nesting:** FOR, GOSUB and WHILE can each be nested up to 32 deep.
- **Files:** 4 open at once; random records up to 1024 bytes; 16 FIELD variables per file.
- **CHAIN:** up to 64 COMMON variables.
- **History:** the last 8 lines typed.
- **The screen:** 80 x 30 as far as BASIC knows (LOCATE, CSRLIN, the KEY line); a bigger terminal window is fine.

12. Example programs

The source's `examples` folder has these; send them over with `XRECV` (section 5) or paste them in.

Program	What it does
PRIMES.BAS	prints the primes below 1000
GUESS.BAS	a guess-the-number game
BOXES.BAS	coloured boxes drawn with the PC line-drawing characters
FILES.BAS	writes, appends and reads back a text file, and stores and reads records (MKI$/CVI) in a random-access file
BLINK.BAS	flashes the LED
FADE.BAS	fades the LED with PWM
ANALOG.BAS	shows the ADC inputs and the chip temperature (GP26-29; change line 16 to `FIRST = 40: LAST = 46` on a Pico Plus 2)
SERIAL.BAS	a small terminal on COM1 (GP8 TX, GP9 RX)
I2CSCAN.BAS	lists the I2C devices on GP2/GP3 and reads the first one's registers 0-7
NEOPIXEL.BAS	red, green, blue, then a rainbow on WS2812B LEDs (the Maker Pi Pico's RGB LED on GP28, or a strip)
SPIFLASH.BAS	reads an SPI flash chip's JEDEC ID and first 16 bytes (SPI0 on GP4-7)

For example, from a shell (terminal closed) and then the terminal:

```
XRECV "PRIMES.BAS"                          (in BASIC)
sx examples/PRIMES.BAS < /dev/ttyACM1 > /dev/ttyACM1
RUN "PRIMES"                                (in BASIC)
```